CARTESIANA — Informe de virtualización

Oct 28, 2026 · @Wenceslao José Villegas

Resumen

El gemelo digital de CARTESIANA funciona de punta a punta. Un programa externo se conecta **directamente por TCP/IP** a la máquina simulada en Webots, que hace de la PC industrial que la máquina tiene a bordo. Antes de trabajar, la máquina hace su **ciclo de inicialización** contra los sensores de inicio de carrera. Después se mueve a mano, se graban puntos y se reproduce la trayectoria. Se probó en Webots R2025a: el robot toma la pieza de un zócalo, la lleva al otro y la devuelve.

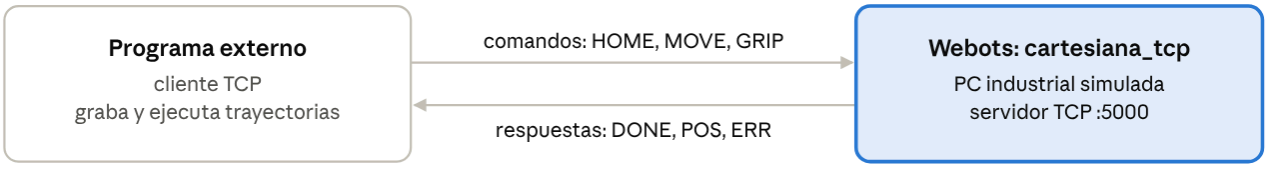
Cambios tras la revisión del 27/10/2026:

- Se eliminó la conexión por puerto serie (COM5 + bridge): la máquina no tiene Arduino sino una PC industrial que escucha por TCP/IP.
- Se agregó la inicialización: la simulación arranca en una posición desconocida y no acepta movimientos hasta buscar los sensores de X, Y y Z.

Se cumplen los requisitos comunes del trabajo final: modelo físico en `worlds/cartesiana_tcp.wbt`, controlador en Python, conexión TCP/IP con mensajes de texto claro regulados por un protocolo propio, y un programa externo nuevo. El modelo cinemático original se mantiene: su validación sigue aprobando 23 de 23 comprobaciones.

Arquitectura

El programa habla directo por TCP/IP con la PC industrial simulada



Socket TCP con líneas de texto, en 127.0.0.1:5000 o en la IP de la PC con Webots. No es una API REST.

comunicación · cliente y servidor TCP

La máquina tiene a bordo una PC industrial que escucha por TCP/IP; en la simulación ese papel lo cumple el controlador de Webots, que es el **servidor**. El programa externo es el **cliente** y se conecta directo, sin puertos serie. Por la conexión viajan líneas de texto: un comando y su respuesta por vez.

Componentes

Pieza	Qué es	Archivo
Robot simulado	Mesa, gabinete, ejes X/Y/Z con motores, sensores de inicio de carrera, grip de dos dedos, dos zócalos de ejemplo y una pieza	`worlds/cartesiana_tcp.wbt`
PC industrial simulada	Controlador de Webots. Servidor TCP en el puerto 5000: recibe órdenes, hace la inicialización, mueve los ejes y el grip, y responde	`controllers/cartesiana_tcp/`
Programa externo	Aplicación de escritorio (Python + Tkinter). Cliente TCP: inicializa, mueve a mano, graba puntos y ejecuta trayectorias	`programa_externo/`
Protocolo	Comandos de texto, una línea por mensaje	`docs/protocolo_cartesiana.md`

El controlador nuevo hereda el controlador cinemático ya validado, así que los límites y las velocidades son los mismos del modelo original.

Inicialización (homing)

La máquina real tiene encoders relativos: al encenderse no sabe dónde está. La simulación reproduce ese comportamiento:

1. Arranca con los ejes en una **posición al azar** y rechaza los movimientos con `ERR NOT_HOMED`.
2. Con `HOME` (botón **Inicializar**), cada eje va primero rápido hacia su sensor de inicio de carrera, se aleja 6 mm y vuelve lento. Cuando el sensor se activa, esa posición es el cero del eje. El orden es **X → Y → Z**.
3. Al terminar, la máquina está en **(0, 0, 0)** y acepta movimientos.

Eje	Sensor	Recorrido después del homing
X	Extremo izquierdo	0 a 497 mm
Y	Extremo delantero	0 a 397 mm
Z	Arriba	−177 a 0 mm

La ubicación de los sensores es un supuesto; si en la máquina están en otro extremo, se cambia en la configuración.

Protocolo

Es texto plano: una línea por mensaje, terminada en salto de línea, con distancias en milímetros de máquina (cero en los sensores). El cliente manda un comando y espera la respuesta antes de mandar el siguiente. No es una API REST: es un socket TCP abierto con mensajes de texto.

Comando	Respuesta	Uso
`PING`	`PONG`	Probar el enlace
`HOME`	`DONE HOME`	Ciclo de inicialización X → Y → Z
`MOVE x y z`	`DONE MOVE x y z`	Mover los tres ejes (requiere HOME)
`GRIP CLOSE`	`DONE GRIP CLOSE HELD` o `EMPTY`	Cerrar el grip; indica si tomó la pieza
`GRIP OPEN`	`DONE GRIP OPEN ZOCALO_B`	Abrir; indica dónde quedó la pieza
`SPEED v`	`OK SPEED …`	Velocidad en mm/s
`WAIT ms`	`DONE WAIT`	Pausa
`POS?`	`POS x y z OPEN EMPTY HOMED`	Posición, grip y si está inicializada
`SWITCH?`	`SWITCH X1 Y0 Z0`	Estado de los sensores de inicio de carrera
`STOP`	`OK STOP`	Frena y descarta lo pendiente

Los errores empiezan con `ERR`: por ejemplo, `ERR NOT_HOMED` si se pide un movimiento sin inicializar, o `ERR RANGE X 999.0` si el destino está fuera del recorrido. En los dos casos, la máquina no se mueve.

Cómo probarlo

1. En Webots, abrir `worlds/cartesiana_tcp.wbt` y presionar **Run**. La consola muestra que el servidor escucha en el puerto 5000.
2. Abrir `programa_externo/iniciar_programa.bat`, dejar IP `127.0.0.1` y puerto `5000`, y presionar **Conectar**. El registro muestra `PONG` y el indicador dice "SIN INICIALIZAR".
3. Presionar **Inicializar**: la máquina busca los sensores de X, Y y Z y queda en (0, 0, 0). El indicador pasa a verde.
4. Mover a mano hasta el cubo, cerrar el grip, **Guardar punto actual**, llevarlo al zócalo azul, guardar y presionar **Ejecutar trayectoria**. También se puede abrir `ciclo_A_B_A.json`.

Si Webots corre en otra PC, en el programa se pone la IP de esa PC y se abre el puerto 5000 en su firewall.

Supuestos y pendientes

Varias medidas no se pudieron relevar en la máquina, así que algunas partes del modelo son estimadas.

- **Sensores de inicio de carrera:** se supuso que X está a la izquierda, Y adelante y Z arriba. Se simulan por la posición del carro, 3 mm antes del tope mecánico.
- **Grip:** se modeló como una pinza de dos dedos con medidas aproximadas. El dosificador de pegamento original no se simula porque ya no se usará.
- **Zócalos de ejemplo:** no existen en la máquina; se agregaron para la demostración. Están sobre columnas porque el recorrido Z provisional no llega a la mesa.
- **Agarre:** al cerrar el grip sobre la pieza, la simulación la mueve junto con la punta (no por fricción). Así el pick & place es estable en Webots.

Para acercar el modelo a la máquina real haría falta relevar:

- [ ] Recorrido útil de los ejes X, Y y Z
- [ ] Ubicación de los sensores de inicio de carrera
- [ ] Forma, medidas y accionamiento del grip